Adaptive RDMA Optimization for Distributed LLM Workloads

Update 8: RDMA Tensor Cache Architecture and Database Design

Nathan Gopee
MS Computer Science Candidate
SUNY New Paltz
gopeen1@newpaltz.edu

Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

February 2026

Abstract

Heterogeneous GPU clusters present a precision mismatch problem that limits distributed LLM training and inference. This update presents the RDMA Tensor Cache, a centralized precision management layer for a two-tower home lab: Tower 1 (Ryzen 9 7950X, RTX 5070 Ti 16GB, Windows) and Tower 2 (Proxmox/Linux, 2× Tesla V100 32GB). We resolve the V100 (FP16-only) vs. 5070 Ti (BF16-native, no kernel RDMA) asymmetry with an RDMA-accessible FP32 master weight store, stochastic rounding, predictive tensor prefetching, and a centralized KV cache eliminating per-rank duplication under MLA/MQA tensor parallelism. We introduce GPU pipeline optimization via Amdahl's and Gustafson's scaling laws, analyze CUDA vs. Triton kernel abstraction hierarchies, and describe vLLM semantic routing for heterogeneous clusters. Two new tools—ScuffedKernels (RDMA-aware kernel optimization) and ScuffedSearch (autonomous RDMA research automation)—provide systematic frameworks for kernel-level performance engineering and experiment tracking. Integration paths through UCX/UCC, SoftRoCEv2, and UEC 1.0 are described.

Contents

1 Overview **4**

2 NVIDIA Communication Stack Summary **4**

3 vLLM Parallelism and MLA/MQA KV Cache Problem **5**
3.1 The MLA/MQA KV Cache Problem 5

4 RDMA Tensor Cache Architecture **6**
4.1 Precision Pipeline 6

4.2 Stochastic Rounding . . . 7
4.3 Centralized KV Cache via vLLM Connector . . . 7

5 Tensor Database Architecture **8**
5.1 Module Mapping . . . 8
5.2 Device Capability Model . . . 8
5.3 Transport Abstraction . . . 10

6 GPU Pipeline Optimization and Scaling Laws **10**
6.1 Amdahl's Law Applied to Kernel Optimization . . . 10
6.2 Gustafson's Law: Workload Scaling . . . 10
6.3 Arc Compute Pipeline Saturation . . . 10

7 CUDA vs Triton: Kernel Abstraction Hierarchy **11**
7.1 CUDA Stack Layers . . . 11
7.2 Triton: Block-Level Kernel Language . . . 11
7.3 NVSHMEM: GPU-to-GPU RDMA . . . 12
7.4 Flash Attention: Triton vs CUDA Implementations . . . 12
7.5 AMD IRIS: Cross-Platform GPU Communication . . . 12

8 Semantic Routing and Cross-Platform GPU Orchestration **12**
8.1 vLLM Semantic Router . . . 12

9 Experimental Design: Three Pillars **13**
9.1 Pillar 1: Precision Switching Overhead . . . 13
9.2 Pillar 2: Prefetching Effectiveness . . . 13
9.3 Pillar 3: Adaptive Quantization . . . 13

10 Fine-Tuning and SAE Feature Steering **14**
10.1 Heterogeneous Training Pipeline . . . 14
10.2 Sparse Auto-Encoder Feature Steering . . . 14

11 ScuffedKernels: RDMA-Aware Kernel Optimization **14**
11.1 Architecture . . . 14
11.2 RDMA Extensions . . . 15
11.3 Baseline Kernels . . . 15

12 ScuffedSearch: Autonomous RDMA Research Automation **15**
12.1 Design Philosophy . . . 16
12.2 Experiment Protocol . . . 16

13 Implementation Status **16**
13.1 Existing Middleware 16
13.2 SoftRoCEv2 Baseline 16
13.3 UCX/UCC Integration Path 17
13.4 Proxmox GPU Passthrough 17

1 Overview

This update advances the thesis in three directions. First, we formalize the RDMA Tensor Cache architecture that bridges the precision gap between V100 and 5070 Ti GPUs by centralizing master weights in the tensor database's `tensor_rdma` crate. Second, we analyze the parallelism strategies available in vLLM and identify a structural limitation of tensor parallelism for MLA/MQA models that motivates our centralized KV cache design. Third, we introduce Sparse Auto-Encoders (SAEs) as an alternative to gradient-based fine-tuning, showing how SAE feature vectors map naturally to the tensor database's `SparseVector` representation with 51× compression.

The home lab consists of two towers connected by Mellanox ConnectX-4 100GbE NICs:

Table 1: Home lab hardware configuration

Component	**Tower 1 (Windows)**	**Tower 2 (Proxmox/Linux)**
CPU	Ryzen 9 7950X (16c/32t)	Xeon (host CPU)
RAM	192 GB DDR5	Host RAM
GPU	RTX 5070 Ti 16 GB (compute 12.0)	2× Tesla V100 32 GB
NIC	Mellanox ConnectX-4 100GbE	Mellanox ConnectX-4 100GbE
OS	Windows 11	Proxmox (Debian/Linux)
RDMA	TCP/gRPC fallback (no kernel RDMA)	SoftRoCEv2 / GPUDirect RDMA
BF16	Native	Not supported
FP16	Supported	Native compute
IP	–	192.168.2.111 (South-Tower)

The fundamental tension is that the V100 is a datacenter GPU with full GPUDirect RDMA support and Tensor Core FP16 compute but no BF16, while the 5070 Ti is a consumer GPU with BF16 native support but no RDMA stack on Windows. The RDMA Tensor Cache resolves this asymmetry by placing FP32 master weights in the tensor database, accessible from both towers through different transport paths.

2 NVIDIA Communication Stack Summary

The NVIDIA communication stack comprises several interdependent libraries. Table 2 summarizes their roles and our usage.

For a tensor of size S bytes, the total transfer time over RDMA is:

$$T_{\text{RDMA}} = T_{\text{latency}} + \frac{S}{B_{\text{link}}} + T_{\text{DMA}} \tag{1}$$

where $T_{\text{latency}} \approx 2$ μs (hardware RoCE), $B_{\text{link}} = 12.5$ GB/s (100GbE), and $T_{\text{DMA}} \approx 0.5$ μs. For a 256 MB gradient tensor, $T_{\text{RDMA}} \approx 20.5$ ms, dominated by bandwidth.

Table 2: NVIDIA communication stack summary

Library	Role	Our Usage
NCCL [16]	GPU collective ops (AllReduce, AllGather)	V100↔V100 on Tower 2; c heterogeneous precision
UCX/HPC-X [15, 20]	Transport-agnostic point-to-point (IB, RoCE, TCP, SHM)	Transport negotiation; he path selection
GPUDirect RDMA [23]	NIC↔GPU DMA via `nvidia-peermem`	V100 direct memory acces
GDRCopy [14]	Low-latency CPU↔GPU copies	Small tensor/metadata tra
NIXL [18]	Point-to-point tensor transfer for disaggregated inference	Reference for vLLM conne
Megatron-LM [25]	Canonical TP/PP/DP parallelism	FP32 master weight patte generalizes this to RDMA

3 vLLM Parallelism and MLA/MQA KV Cache Problem

The choice between tensor parallelism (TP) and pipeline parallelism (PP) determines RDMA communication patterns. Table 3 summarizes the trade-offs for our cluster.

Table 3: Parallelism strategy comparison for the two-tower cluster

Property	Tensor Parallelism	Pipeline Parallelism
Communication pattern	AllReduce per layer	Activation transfer per stage
Communication frequency	$2L$ per forward pass	$p-1$ per micro-batch
Latency sensitivity	High (synchronous)	Low (async pipeline)
Suitable interconnect	NVLink, PCIe	Ethernet, RDMA
Our intra-Tower 2	V100↔V100 (PCIe)	Not needed (2 GPUs)
Our cross-tower	Not recommended	Tower 1↔Tower 2

Our optimal strategy is TP=2 within Tower 2 (V100 pair over PCIe) combined with PP across towers (100GbE RDMA). The 5070 Ti handles a separate pipeline stage.

3.1 The MLA/MQA KV Cache Problem

Multi-Latent Attention (MLA, DeepSeek-V2 [7]) and Multi-Query Attention (MQA) share a single key-value head across all attention heads, creating a structural incompatibility with tensor parallelism: the KV cache cannot be sharded along the head dimension. Under TP, each rank stores a *full copy*:

$$\text{KV Memory (per rank, MQA)} = 2 \cdot L \cdot C \cdot d_{\text{kv}} \quad \text{(no reduction with TP)} \tag{2}$$

Challenge 1. MLA and MQA models cannot benefit from tensor parallelism for KV cache distribution. Each TP rank stores a full copy, negating the memory savings these attention variants provide.

Our solution externalizes the KV cache to the tensor database, accessed via RDMA. A single

centralized copy serves all TP ranks through one-sided RDMA reads, reducing total KV memory from TP $\times\, 2LC \cdot d_{\text{kv}}$ to $2LC \cdot d_{\text{kv}} + \text{TP} \times \text{prefetch_buffer}$.

4 RDMA Tensor Cache Architecture

The RDMA Tensor Cache mediates all tensor transfers between Tower 1 and Tower 2, handling precision conversion, gradient accumulation, and KV cache management. Figure 1 shows the overall system architecture.

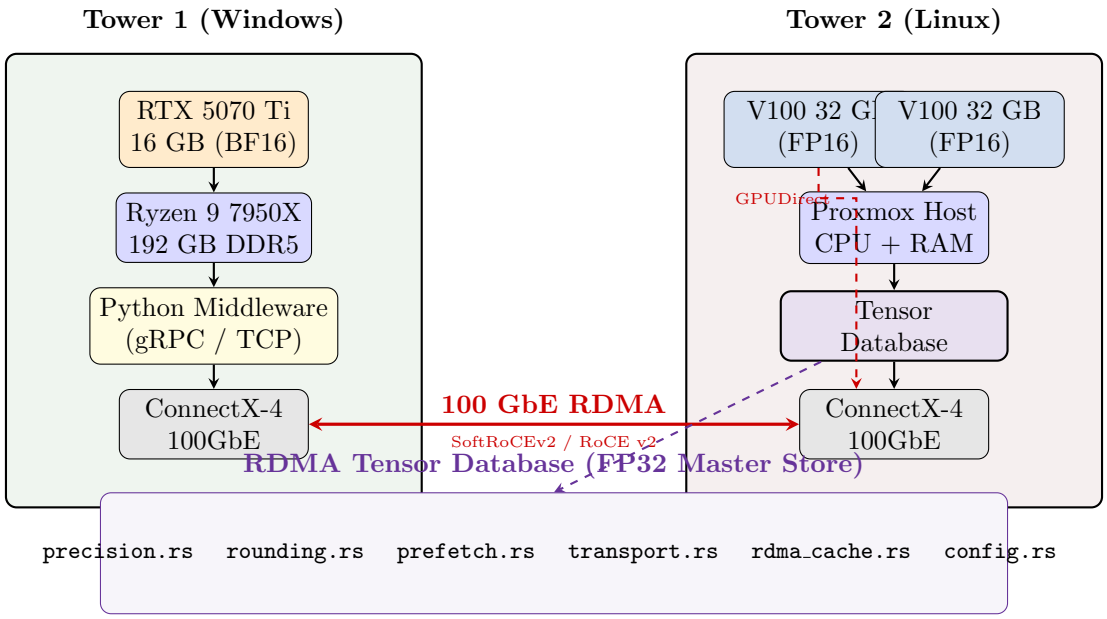

Figure 1: Two-tower system architecture. The tensor database's `tensor_rdma` crate runs on Tower 2, serving as the centralized FP32 master weight store. Tower 1 accesses tensors via gRPC/TCP fallback; Tower 2's V100s access them via GPUDirect RDMA through `nvidia-peermem`.

4.1 Precision Pipeline

FP16 (V100 Tensor Cores) has 5 exponent bits and range $[6 \times 10^{-8},\ 65504]$; gradients routinely exceed this range. BF16 shares FP32's 8 exponent bits but is unavailable on V100. The RDMA Tensor Cache resolves this through a three-stage precision pipeline (Figure 2): V100 FP16 gradients $\rightarrow$ FP32 master weights in tensor database $\rightarrow$ stochastic rounding to device-native format (BF16 for 5070 Ti, FP16 for V100).

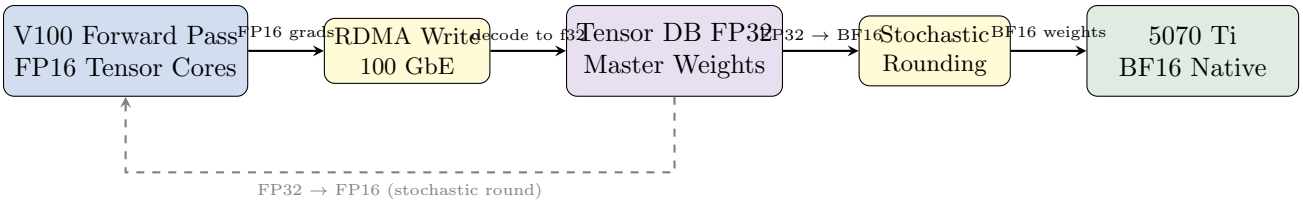

Figure 2: Precision pipeline. Gradients computed in V100 FP16 are accumulated into FP32 master weights. Stochastic rounding converts to device-native precision for each GPU.

4.2 Stochastic Rounding

Stochastic rounding eliminates the systematic bias of round-to-nearest-even. For an FP32 value x rounded to a lower precision with adjacent representable values $\lfloor x \rfloor$ and $\lceil x \rceil$:

$$P(\text{round up}) = \frac{x - \lfloor x \rfloor_{\text{fp16}}}{\lceil x \rceil_{\text{fp16}} - \lfloor x \rfloor_{\text{fp16}}} \tag{3}$$

giving the unbiasedness property $\mathbb{E}[\text{round}(x)] = x$, ensuring gradient accumulation converges to the correct value despite per-step precision loss [12]. The implementation in `rounding.rs` uses the `half` crate for FP16/BF16 manipulation and uniform random draws to decide rounding direction.

4.3 Centralized KV Cache via vLLM Connector

The MLA/MQA duplication problem (Section 3.1) is resolved by externalizing the KV cache to the tensor database via a vLLM V1 KV Connector. vLLM's connector architecture [28] separates KV cache management into scheduler-side and worker-side methods, enabling pluggable backends (NIXL [18], Mooncake [13], P2P NCCL). Our `RDMAKVConnector` implements this interface to back the KV cache with the RDMA tensor database.

Figure 3 shows the data flow. vLLM V1 supports combining HBM Prefix Caching with an external KV Pool via the KV Connector V1 interface [28]. The lookup proceeds in two tiers:

1. **HBM prefix cache**: The engine first checks for prefix hits in local GPU HBM (enabled by default in vLLM V1 unless `--no_enable_prefix_caching` is set).

2. **RDMA KV Pool**: After determining HBM hits, the connector queries the tensor database for additional matches. Only the blocks *not* found in HBM are fetched from the KV Pool via RDMA, minimizing transfer latency.

3. **Resume**: Once KV Pool blocks are loaded into HBM, the remaining process is identical to standard HBM Prefix Caching—no special handling is needed downstream.

For disaggregated prefill/decode (PD) serving, the architecture extends with vLLM's `MultiConnector` [28]: prefill nodes use the RDMA KV Pool connector for prefix caching and writeback, while decode nodes receive KV caches via a P2P connector (direct device-to-device transfer). This preserves

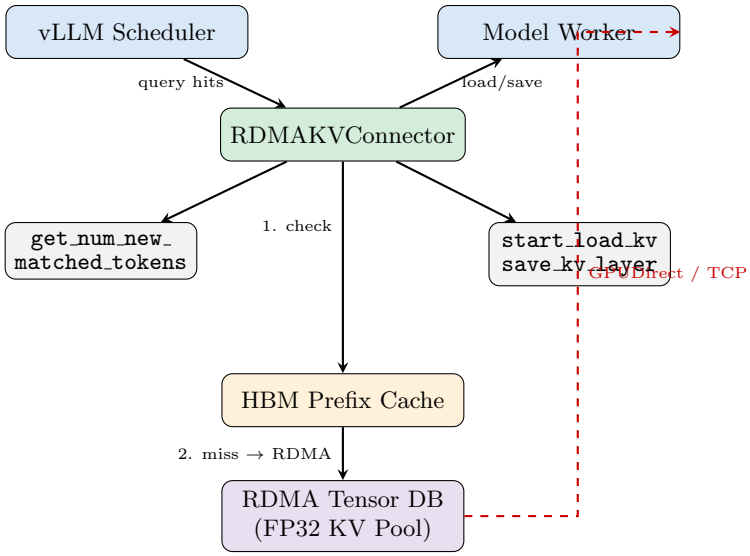

Figure 3: RDMA KV Connector data flow. The scheduler queries prefix hits, the connector checks HBM first then fetches missing blocks from the tensor database via RDMA, and the worker receives KV data without maintaining per-rank copies.

prefix caching gains on prefill nodes without sacrificing P2P transfer efficiency to decode nodes. In our cluster, Tower 2 V100s handle prefill (writing KV to the tensor database), and the 5070 Ti on Tower 1 handles decode (reading via TCP/gRPC since it lacks kernel RDMA).

The connector's scheduler-side method `get_num_new_matched_tokens` returns how many tokens have cached KV entries, allowing the engine to skip redundant prefill computation. Worker-side `start_load_kv` initiates async RDMA reads for the needed blocks, and `save_kv_layer` writes new KV data back per-layer as prefill progresses.

5 Tensor Database Architecture

The `tensor_rdma` crate [9] depends on `tensor_store` for local key-value storage and `tensor_compress` for delta encoding. Figure 4 shows the dependency structure.

5.1 Module Mapping

Table 4 maps thesis concepts to their Rust implementations.

5.2 Device Capability Model

The `DeviceCapability` struct encodes GPU capabilities, and `PrecisionRouter` selects wire formats. The V100 preset supports FP16/FP32 with GPUDirect RDMA; the 5070 Ti preset supports FP16/BF16/FP32 without RDMA. The cache stores in FP32 and converts per-device, avoiding the precision ceiling a common format would impose.

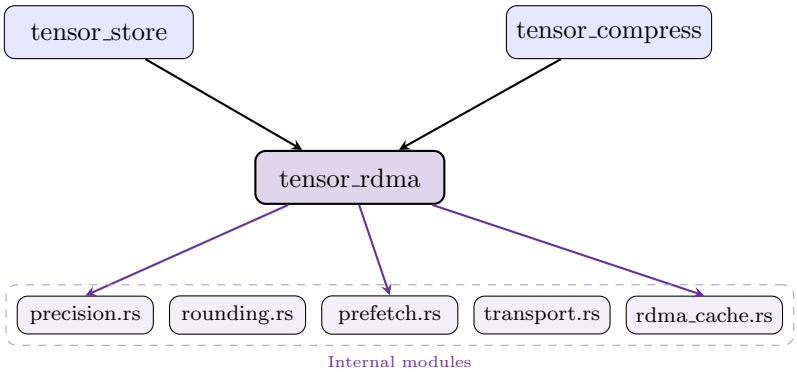

Figure 4: `tensor_rdma` crate: depends on `tensor_store` and `tensor_compress`, with five internal modules implementing the RDMA Tensor Cache.

Table 4: Thesis concepts mapped to `tensor_rdma` modules

Thesis Concept	**Module**	**Key Types / Functions**
Precision management	`precision.rs`	`PrecisionFormat`, `DeviceCapability`, `PrecisionRouter`, `convert_precision`
Stochastic rounding	`rounding.rs`	`stochastic_round_f32_to_f16`, `apply_adam_update`, `apply_gradient_batch`
Predictive prefetching	`prefetch.rs`	`PrefetchEngine`, `AccessPattern`, `classify_pattern`
RDMA transport	`transport.rs`	`RdmaTransport` trait, `MemoryTransport`, `TcpFallbackTransport`
Tensor cache	`rdma_cache.rs`	`RdmaTensorCache<T>`, `put_tensor`, `get_tensor`, `apply_gradient`
Configuration	`config.rs`	`RdmaCacheConfig`, `PrefetchStrategy`, `TransportConfig`

5.3 Transport Abstraction

The `RdmaTransport` trait defines four core operations: `register_buffer` (analogous to `ibv_reg_mr`), `deregister_buffer`, `rdma_write` (one-sided write), and `rdma_read` (one-sided read). Three implementations exist: `MemoryTransport` for testing, `TcpFallbackTransport` for Tower 1, and the planned `SoftRoceTransport` for Tower 2.

6 GPU Pipeline Optimization and Scaling Laws

Optimizing GPU kernels for RDMA-aware workloads requires principled prioritization. We apply classical scaling laws to determine which kernels yield the greatest whole-program speedup.

6.1 Amdahl's Law Applied to Kernel Optimization

Amdahl's Law [2] gives the maximum speedup when accelerating a fraction p of execution by factor n:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} \tag{4}$$

This directly informs kernel prioritization: a kernel consuming 40% of GPU time ($p = 0.4$) yields at most $S(\infty) = 1/(1-0.4) = 1.67\times$ whole-program speedup, regardless of how fast the kernel becomes. Conversely, a kernel at 5% ($p = 0.05$) caps at $1.05\times$. ScuffedKernels (Section 11) uses this to rank optimization targets by their *Amdahl priority*: the product of potential speedup and absolute time saved.

6.2 Gustafson's Law: Workload Scaling

Gustafson's Law [10] provides the complementary perspective for scaled workloads:

$$S(n) = n - \alpha(n-1) \tag{5}$$

where α is the serial fraction. For RDMA pipelines, increasing batch size scales the parallel portion while the serial overhead (connection setup, QP initialization) remains fixed. As a general example, with $\alpha = 0.05$ and $n = 4$ processors, $S(4) = 4 - 0.05 \times 3 = 3.85\times$—near-linear scaling. For our 3-GPU cluster, the applicable scaling depends on the measured serial fraction of the RDMA pipeline, which has not yet been determined.

6.3 Arc Compute Pipeline Saturation

Task matching for seamless GPU processing requires that compute kernels and RDMA transfers are pipelined such that neither the GPU nor the network is idle. The pipeline throughput is bounded by:

$$T_{\text{pipeline}} = \max(T_{\text{compute}}, T_{\text{transfer}}) \tag{6}$$

When $T_{\text{compute}} \approx T_{\text{transfer}}$, the pipeline is saturated and both resources are fully utilized. ScuffedKernels achieves this by matching kernel optimization effort to the transfer-limited ceiling: optimizing a kernel below T_{transfer} yields no further benefit.

7 CUDA vs Triton: Kernel Abstraction Hierarchy

GPU kernel development spans multiple abstraction levels. We use Triton for ScuffedKernels development, with CUDA paths planned for performance-critical kernels.

7.1 CUDA Stack Layers

CUDA provides four levels: Driver API (`cuLaunchKernel`, direct GPU control), Runtime API (`cudaLaunchKernel`, simplified launch), CCCL (CUB/Thrust template libraries), and C++ kernels (thread-level programming with explicit indexing and shared memory). CUDA Python wraps the Driver/Runtime APIs but cannot express kernels—Numba CUDA or Triton is required for kernel authoring.

7.2 Triton: Block-Level Kernel Language

Triton [26] raises the abstraction from individual threads to *blocks* of data. Key differences from CUDA:

- **Block-level programming**: Operations on tiles (e.g., `tl.load`, `tl.dot`) rather than per-thread indexing.
- **Automatic memory management**: Triton handles shared memory allocation and bank conflicts.
- **Python-like syntax**: Kernels are decorated Python functions (`@triton.jit`).
- **Auto-tuning**: `triton.autotune` searches block sizes, warps, and pipeline stages.

The Triton compiler uses a multi-stage pipeline [3]: Triton IR (TTIR) → TritonGPU IR (TTGIR, with GPU-specific optimizations) → LLVM IR (LLIR) → backend-specific output (PTX/CUBIN for NVIDIA, AMDGCN/HSACO for AMD). Compiled kernels are cached under `~/.triton/cache` keyed by source, parameters, and environment, enabling rapid iteration. Cross-architecture compilation via the `GPUTarget` API (e.g., `GPUTarget("cuda", 80, 32)` for SM_80) permits artifact inspection without target hardware. For AMD targets, the compiled `.amdgcn` files expose register counts, spill information, and occupancy metrics directly.

Our ScuffedKernels baseline kernels (matmul, softmax, attention) are implemented in Triton for development speed, with CUDA fallback paths planned for performance-critical paths.

7.3 NVSHMEM: GPU-to-GPU RDMA

NVSHMEM [17] provides OpenSHMEM-like put/get semantics for GPU-to-GPU communication. Unlike NCCL (collective-focused), NVSHMEM supports one-sided operations from GPU kernels via `nvshmem_float_put` and `nvshmem_float_get`. However, NVSHMEM initialization is host-side only, and our Windows tower cannot participate in the symmetric heap. NVSHMEM is relevant for V100↔V100 transfers within Tower 2.

7.4 Flash Attention: Triton vs CUDA Implementations

Flash Attention [6] reduces attention memory from $O(N^2)$ to $O(N)$ by tiling Q, K, V into SRAM-resident blocks and using the online softmax trick—maintaining running (m, ℓ) statistics and rescaling partial results per block:

$$m_{\text{new}} = \max(m_{\text{prev}}, \tilde{m}), \quad \ell_{\text{new}} = e^{m_{\text{prev}} - m_{\text{new}}} \cdot \ell_{\text{prev}} + e^{\tilde{m} - m_{\text{new}}} \cdot \tilde{\ell} \tag{7}$$

Block sizes are chosen as $B_c = \lceil M/(4d) \rceil$ (SRAM capacity M, head dimension d).

A Triton implementation study [8] shows the optimization progression: naive (166 ms) → register accumulation (157 ms) → K-transpose to fix 6.3-way shared memory bank conflicts (34 ms, 4.9× total). The remaining bottleneck is MIO pipeline stalls from `exp()`/`max()` (44–50% of time). Flash Attention 4 [5] addresses this on Blackwell with warp specialization (producer/consumer warps), reporting 1.4 PFLOPS on B200. This illustrates Tier 5 of our optimization playbook (Section 11).

7.5 AMD IRIS: Cross-Platform GPU Communication

AMD's IRIS framework [1] brings SHMEM-like remote memory access (RMA) to ROCm GPUs within Triton kernels. IRIS is relevant for future cluster heterogeneity—if AMD GPUs join the cluster, IRIS provides a Triton-native communication path analogous to NVSHMEM. Our transport abstraction is designed to accommodate such backends.

8 Semantic Routing and Cross-Platform GPU Orchestration

8.1 vLLM Semantic Router

The vLLM Semantic Router [29] classifies requests by semantic signal and routes to the best-suited model—optimizing for quality, not just throughput. For our cluster, this enables two-layer routing:

1. **Semantic layer**: Route by task type (code, creative, QA) to the appropriate model.
2. **Latency layer**: Route to the GPU with lowest predicted latency—5070 Ti BF16 for small batches (no precision conversion), V100 FP16 for large batches (RDMA-backed KV cache).

The router's latency predictor can factor in RDMA transfer time from the tensor database, accounting for the different precision pipeline costs on each tower.

9 Experimental Design: Three Pillars

Note: All targets below are **planned/untested**. No experimental results have been collected yet.

We structure validation around three experimental pillars, each targeting a distinct aspect of the RDMA Tensor Cache.

9.1 Pillar 1: Precision Switching Overhead

Hypothesis: Stochastic rounding eliminates gradient bias with acceptable overhead compared to deterministic truncation.

Metrics: Conversion throughput (GB/s) for FP16↔FP32 and FP32↔BF16; statistical bias magnitude for stochastic vs. deterministic; overhead ratio $T_{\text{stochastic}}/T_{\text{deterministic}}$.

9.2 Pillar 2: Prefetching Effectiveness

Hypothesis: Adaptive prefetch improves cache hit rates for both sequential inference and fine-tuning access patterns.

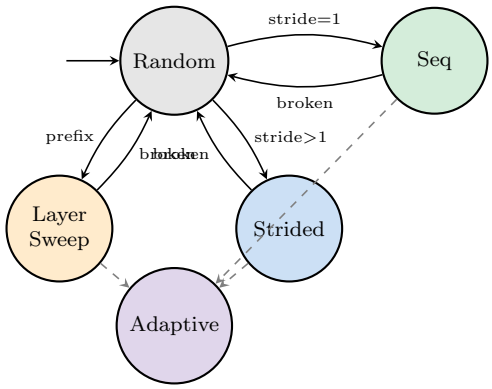

Figure 5: Prefetch engine state machine. Detected patterns delegate to the Adaptive strategy.

Metrics: Hit rate by strategy; latency breakdown (hit vs. miss, SoftRoCE vs. hardware RoCE); wasted prefetch ratio.

9.3 Pillar 3: Adaptive Quantization

Hypothesis: MXFP4 [24] reduces wire bandwidth for KV cache transfers with acceptable accuracy loss.

Metrics: Perplexity delta ΔPPL for FP16/MXFP4 vs. FP32; bandwidth savings per training step; convergence rate.

10 Fine-Tuning and SAE Feature Steering

10.1 Heterogeneous Training Pipeline

The pipeline assigns roles by GPU capability: V100 pair runs forward/backward in FP16, the tensor database accumulates gradients into FP32 master weights, and the 5070 Ti receives BF16 weights for inference validation. This mirrors the Hetis approach [21], which reported 2.25× throughput by assigning distinct pipeline stages to different GPU types.

10.2 Sparse Auto-Encoder Feature Steering

SAEs [4] decompose activation $\mathbf{a} \in \mathbb{R}^d$ into sparse features $\mathbf{f} \in \mathbb{R}^D$ ($D \gg d$):

$$\mathbf{f} = \mathrm{ReLU}(W_{\mathrm{enc}} \cdot \mathbf{a} + \mathbf{b}_{\mathrm{enc}}), \quad \hat{\mathbf{a}} = W_{\mathrm{dec}} \cdot \mathbf{f} + \mathbf{b}_{\mathrm{dec}} \tag{8}$$

Feature steering clamps feature i to $\alpha \cdot \mathbf{f}_{\max,i}$, modifying behavior at inference time without gradient computation [22].

SAE feature vectors map naturally to the tensor database's `SparseVector`: 65,536 dimensions with ∼50 active features yields ∼400 bytes vs. 256 KB dense—51× compression, fitting within a single RDMA read.

Table 5: Fine-tuning approach comparison

Property	**Gradient-Based (LoRA)**	**SAE Steering**
Requires backward pass	Yes	No
GPU memory for optimizer	2–3× model size	Zero
Training time	Hours to days	Seconds (feature lookup)
Cross-tower RDMA traffic	Gradients per step	Feature vectors once
Reversibility	Requires checkpoint	Immediate ($\alpha = 0$)

11 ScuffedKernels: RDMA-Aware Kernel Optimization

ScuffedKernels is our kernel optimization framework, providing systematic profiling, optimization, and validation of GPU kernels with RDMA-aware extensions. The implementation resides in `middleware/scuffedkernels/`.

11.1 Architecture

The system follows a four-stage pipeline:

1. **Profiler** (`profiler.py`): Uses PyTorch CUDA profiler to extract per-kernel timing, then ranks kernels by Amdahl's Law priority (Equation (4)). Kernels are scored by (speedup potential) × (absolute time) to identify high-impact optimization targets.

2. **Optimizer** (`optimizer.py`): Applies a 6-tier playbook, one tier at a time:

   T1: Block/grid dimensions (SM occupancy)

   T2: Memory access patterns (shared memory tiling, coalescing)

   T3: Precision (TF32, FP16 Tensor Cores)

   T4: Persistent kernels (fuse multi-stage into single launch)

   T5: Architecture-specific (warp specialization, SM count tuning)

   T6: RDMA-aware (compute-transfer overlap, prefetch depth)

3. **Benchmarker** (`benchmarker.py`): 5-stage validation (smoke → shapes → stability → determinism → edge cases) followed by performance measurement. Includes RDMA-aware roofline analysis with a network bandwidth ceiling alongside the traditional compute and memory ceilings.

4. **Orchestrator** (`orchestrator.py`): State machine driving the loop: profile → rank → optimize → validate → accept/revert. Each tier is validated before proceeding; failed tiers are reverted.

11.2 RDMA Extensions

ScuffedKernels extends the classical roofline model with a *network ceiling*. For RDMA-bound kernels (those waiting on network transfers), the achievable performance is limited by:

$$P_{\text{net}} = \text{AI} \times B_{\text{network}} \tag{9}$$

where AI is arithmetic intensity (FLOPs/byte) and B_{network} is network bandwidth (12.5 GB/s for 100GbE). Kernels falling below this ceiling are network-bound, and compute optimization alone cannot help—prefetch depth and transfer overlap (Tier 6) are the appropriate interventions.

11.3 Baseline Kernels

Three baseline Triton kernels (`kernels/`) provide benchmarking targets: tiled matrix multiplication, online-normalized softmax, and Flash Attention-style tiled attention with the online softmax trick. Each has a NumPy reference implementation for correctness validation.

12 ScuffedSearch: Autonomous RDMA Research Automation

ScuffedSearch provides a structured framework for systematic RDMA optimization experiments. The implementation resides in `middleware/scuffedsearch/`.

12.1 Design Philosophy

The core principle is *one GPU, one file, one metric*:

- `prepare.py` (IMMUTABLE): Defines the network topology (frozen dataclass), benchmark harness, and scoring function. Agents cannot modify this file—it ensures measurement consistency across experiments.
- `optimize.py` (MUTABLE): The only file agents modify. Contains tunable hyperparameters (batch size, QP depth, prefetch depth, wire format, transport mode, buffer pool size, congestion window) and the `optimize_rdma_transfer()` function.
- `analysis.py`: Tracks experiment outcomes (KEEP/DISCARD/CRASH), computes parameter contribution analysis, and maintains a JSON-serializable experiment history.

12.2 Experiment Protocol

Each experiment follows: (1) make one change to `optimize.py`, (2) run the harness across transfer sizes 1 KB–16 MB, (3) score via score $= \sqrt{\text{throughput_efficiency} \times \text{latency_factor}}$ (geometric mean rewarding both bandwidth and latency), (4) KEEP if >1% improvement, else DISCARD, (5) git commit for reproducibility.

13 Implementation Status

13.1 Existing Middleware

The Python middleware (`middleware/`) implements TCP, SoftRoCE, and TTPoe transports via `TransportSelector`, and the RDMA tensor cache (`PrecisionManager` with stochastic rounding, `PrefetchEngine`, adaptive quantization). Tested against the `MemoryTransport` in-process mock.

13.2 SoftRoCEv2 Baseline

Status: Projected targets, not yet measured.

Table 6: SoftRoCEv2 vs. Hardware RoCE: published specifications (not our measurements)

Metric	SoftRoCEv2 (typical)	Hardware RoCE (spec)	Ratio
Latency (1 byte)	∼190 μs	∼2 μs	95×
Bandwidth (large msg)	∼8 Gbps	∼90 Gbps	11×
CPU overhead	High (software path)	Near-zero (NIC offload)	–
GPUDirect RDMA	Via host bounce	Direct NIC→GPU	–

SoftRoCEv2 is configured on Tower 2 via the `rdma_rxe` kernel module, creating an RXE device bound to the physical NIC. It exercises the same verbs API as hardware RoCE, so code developed against it requires no changes at transition.

13.3 UCX/UCC Integration Path

Table 7: Middleware transport mapping to UCX

Middleware Transport	UCX Transport	Use Case
`TcpFallbackTransport`	`tcp`	Tower 1 (Windows)
`SoftRoceTransport`	`rc` (over rxe0)	Development baseline
Hardware RoCE (planned)	`rc` or `dc` (mlx5_0)	Production 100GbE
GPUDirect RDMA	`cuda_copy,gdr_copy`	V100 GPU memory

UCC [19] provides collective operations on top of UCX point-to-point primitives. A recent vLLM RFC [30] proposes UCC as an alternative to NCCL. For our cluster, UCX transport negotiation handles the heterogeneous path automatically, and UEC 1.0 [27] may provide native RDMA-optimized Ethernet via ConnectX firmware updates.

13.4 Proxmox GPU Passthrough

Tower 2 runs Proxmox (QEMU/KVM), with V100 GPUs passed through via VFIO-PCI. QEMU device properties can be overridden via libvirt's `qemu:override` XML element [11], which supports per-device frontend property tuning (string, unsigned, signed, bool, or remove). This is relevant for configuring PCIe BAR sizes and IOMMU settings that affect GPUDirect RDMA performance—incorrect BAR mapping can force host-bounce transfers instead of direct NIC→GPU DMA.

References

[1] AMD. IRIS: ROCm multi-GPU communication with SHMEM-like RMA in Triton. https://github.com/ROCm/triton, 2025.

[2] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

[3] Lei Chen. Triton compiler development tips. https://www.lei.chat/posts/triton-compiler-development-tips/, 2024.

[4] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[5] Tri Dao and Jay Shah. FlashAttention-4: Hardware-friendly again with warp specialization on Blackwell GPUs. https://tridao.me/blog/2025/flash4/, 2025.

[6] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[7] DeepSeek AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[8] Amine Diro. Flash attention from scratch in Triton. https://aminediro.com/posts/flash_attn/, 2024.

[9] Nathan Gopee. Scuffedrdma: Adaptive rdma optimization for distributed llm workloads. https://github.com/ndg8743/ScuffedRDMA, 2025.

[10] John L. Gustafson. Reevaluating Amdahl's law. *Communications of the ACM*, 31(5):532–533, 1988.

[11] libvirt Contributors. QEMU/KVM hypervisor driver: Overriding properties of QEMU devices. https://libvirt.org/drvqemu.html#overriding-properties-of-qemu-devices, 2025.

[12] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. Mixed precision training. *International Conference on Learning Representations (ICLR)*, 2018.

[13] Moonshot AI. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.

[14] NVIDIA. GDRCopy: A low-latency GPU memory copy library. https://github.com/NVIDIA/gdrcopy, 2024.

[15] NVIDIA. HPC-X v2.15 documentation. https://docs.nvidia.com/networking/display/hpcxv215, 2024.

[16] NVIDIA. NCCL: NVIDIA collective communications library. https://github.com/NVIDIA/nccl, 2024.

[17] NVIDIA. NVSHMEM: NVIDIA OpenSHMEM library for GPUs. https://developer.nvidia.com/nvshmem, 2024.

[18] NVIDIA. NIXL: NVIDIA inference xfer library. https://github.com/NVIDIA/nixl, 2025.

[19] OpenUCX Consortium. UCC: Unified collective communication. https://github.com/openucx/ucc, 2024.

[20] OpenUCX Consortium. UCX: Unified communication X. https://github.com/openucx/ucx, 2024.

[21] B. Peng et al. Hetis: Heterogeneous GPU-cluster scheduling for efficient large language model fine-tuning. *Proceedings of the 2024 USENIX Annual Technical Conference*, 2024.

[22] J. Philipp. Eiffel tower Llama: SAE feature steering demo. https://huggingface.co/spaces/janphilipp/eiffel_tower_llama, 2024.

[23] Red Hat. GPUDirect RDMA on OpenShift AI: Enabling remote GPU memory access for accelerated computing. https://www.redhat.com/en/blog/gpudirect-rdma-openshift-ai-enabling-remote-gpu-memory-access-for-accelerated-computing, 2024.

[24] B. D. Rouhani et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[25] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2020.

[26] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.

[27] Ultra Ethernet Consortium. Ultra Ethernet Consortium 1.0 specification, 2025.

[28] vLLM Contributors. KV cache pool guide: Combining HBM prefix caching with Mooncake PD disaggregation. https://docs.vllm.ai/projects/ascend/en/latest/developer_guide/feature_guide/KV_Cache_Pool_Guide.html, 2025.

[29] vLLM Contributors. Semantic router: Signal-driven mixture-of-models routing. https://github.com/vllm-project/vllm, 2025.

[30] vLLM Contributors. RFC: UCC integration for collective communication (PR #30930). https://github.com/vllm-project/vllm/pull/30930, 2025.